RAG Assignment: Baseline vs. Re-Ranking

Document: OSHA 29 CFR 1910 – General Industry Special Requirements (2026), 378 pages, 757 chunks.
Setup: bi-encoder `all-MiniLM-L6-v2` + FAISS (inner product on normalised vectors); cross-encoder `ms-marco-MiniLM-L-6-v2`; LLM `gemini-3.5-flash-lite`, temperature 0, same prompt for both pipelines.
Code: `https://github.com/Rahul99Saha/RAG-Assignment`

1. Architecture Comparison

A bi-encoder turns the query and each chunk into a vector separately, so the chunk vectors can be computed once and stored; at query time we only embed the query and take cosine similarity. The model never sees the query and chunk together, so it can only compare their overall "topics". A cross-encoder puts the query and one chunk into the same input and runs self-attention across all their tokens, so words in the query can directly interact with words in the chunk (for example "not" or "except"). That is more accurate, but nothing can be precomputed: every (query, chunk) pair needs its own forward pass.

One more thing I found: the bi-encoder cuts input at 256 tokens, and 569 of my 757 chunks (75%) are longer than that. So an "except…" sentence near the end of a chunk is never seen by the bi-encoder, while the cross-encoder reads up to 512 tokens.

2. Qualitative and Rank Shift Analysis

For each query I read the PDF to find the chunk that really contains the rule (the gold chunk), then compared its rank before and after re-ranking.

Query (failure type)	Gold chunk	Baseline	Re-ranked
Fall protection while assessing conditions before work (exception)	C0014	#4	#2
Confined-space attendant doing other duties (exception)	C0412	#4	#1
Beard and tight-fitting respirator (negation/constraint)	C0160 / C0162	#8 / #6	#1 / #3
Who may remove another worker's lock (role limit)	C0209	#2	#1
Small company, written fire prevention plan (keyword trap)	C0034	#1	#2
Hydrostatic testing of extinguishers (keyword trap)	C0224	#1	#1

The gold chunk was in the top 3 for 3 of 6 queries with the baseline and 6 of 6 after re-ranking. MRR@10 went from 0.53 to 0.83. In the first three queries the key chunk was outside the top 3 at the start (ranks 4–8) and was promoted into it.

Answer 1 vs. Answer 2.

- *Fall protection:* Answer 1 said "the provided context does not answer this". The exception in 1910.28(a)(2)(ii) was at rank 4, just outside the top 3. Answer 2 stated the exception correctly. This is the clearest case where the missing chunk caused a non-answer.
- *Lock removal:* Both answers were correct, but Answer 1 only gave the exception (device removed under the employer's direction) and left out the main rule that the person who

applied the lock removes it. Answer 2 gave both. Answer 2 also added a rule from a different standard (1910.269, system operator), which is a distractor that re-ranking let in.

- *Fire prevention plan:* Both answers correctly said employers with 10 or fewer employees can communicate the plan orally. Answer 1 also said the plan must be written otherwise; Answer 2 omitted this. Re-ranking did not help here: it moved an unrelated chunk (C0645) from #6 to #1.
- *Attendant, beard, extinguisher testing:* both answers were "context does not answer this". For the beard query the regulation never mentions beards (it only requires fit testing and no seal leakage), so this answer is honest. For the attendant query the rule is split across two chunks (C0411 and C0412) and the model did not combine them. For the extinguisher query the test interval was not in the retrieved text. So re-ranking improved the ranking here, but not the final answer.

None of the answers contained a clear hallucination; the baseline's failure mode was giving no answer or leaving out a clause, not inventing one.

3. Latency Profiling

Median of 20 runs per query (after warm-up), on my laptop CPU.

Stage	Median time
Pipeline 1: embed query + FAISS top-10	4.9 ms
Pipeline 2: cross-encoder on 10 pairs	73.9 ms
Pipeline 2: total retrieval (1 + 2)	79.6 ms
LLM generation (one answer)	1.41 s

Re-ranking made retrieval about 15–20× slower (5 ms → 80 ms), but it is small next to the LLM call, so the end-to-end cost is only a few percent more.

4. Scaling Question

A cross-encoder needs a forward pass for every (query, chunk) pair, and it depends on the query, so nothing can be computed in advance. On my machine it costs about 6.1 ms per pair. Scoring all 757 chunks took 4.65 s, compared with 7 ms for the bi-encoder search. Extrapolated to a million chunks that is about 6,100 s (roughly 1.7 hours) per query, which is not usable. The bi-encoder embeds the chunks once offline and then only needs a fast nearest-neighbour search, so it is used to narrow the collection to a few candidates, and the slower, more accurate cross-encoder only re-orders those few.